

Aula 05 — Construtores e estado inicial válido

Se um objeto precisa de certas informações para fazer sentido, por que permitimos que ele seja criado incompleto?

TRAJETÓRIA

Hoje vamos investigar

objeto incompleto → criação inicializada → estado inicial coerente

Ao final do Laboratório 04...

```
ItemPedido item = new ItemPedido();  
item.descricao = "Teclado";  
item.precoUnitario = 150.0;  
item.aumentarQuantidade(2);
```

ATIVIDADE

O OBJETO INCOMPLETO

Pare depois da primeira linha

```
ItemPedido item = new ItemPedido();
```

Qual é o estado do objeto nesse momento?

- `descricao` = ?
- `precoUnitario` = ?
- `quantidade` = ?

O objeto já existe

Campo	Valor nesse momento
descricao	null
precoUnitario	0.0
quantidade	0

Existir para Java ainda não significa representar o item que queremos usar.

null e valores padrão

- campos numéricos começam em `0` ou `0.0` ;
- campos que guardam referências começam em `null` ;
- `String` é um tipo de referência;
- `null` indica que a referência não aponta para um objeto.

Quando o item fica pronto?

```
ItemPedido itemA = new ItemPedido();  
itemA.descricao = "Teclado";  
itemA.precoUnitario = 150.0;  
itemA.aumentarQuantidade(2);
```

E se uma etapa for esquecida?

```
ItemPedido itemA = new ItemPedido();  
itemA.descricao = "Teclado";  
itemA.precoUnitario = 150.0;  
itemA.aumentarQuantidade(2);
```

```
ItemPedido itemB = new ItemPedido();  
itemB.descricao = "Mouse";  
itemB.aumentarQuantidade(3);
```

1. Qual objeto ficou incompleto?
2. O que `itemB.calcularSubtotal()` produz?
3. A linguagem percebe que faltou uma etapa?

O segundo trecho compila

Preço

0.0

permaneceu no valor padrão

Quantidade

3

foi alterada depois da criação

Subtotal

0.0

é calculado com esse estado

Java conhece os tipos. A expectativa de que o item esteja completo pertence ao domínio.

Criar agora, completar depois

- uma etapa pode ser esquecida;
- outro método pode receber o objeto cedo demais;
- cada cliente precisa conhecer a sequência de preparação;
- diferentes clientes podem preparar o mesmo tipo de maneiras incompatíveis.

CONCEITO-CHAVE

O PROBLEMA IDENTIFICADO

Estado inicial

É o conjunto de valores que um objeto possui quando sua criação termina.

Proteger mudanças posteriores não basta quando a criação ainda permite estados inadequados ou incompletos.

UMA NOVA NECESSIDADE

Se já sabemos do que o item precisa...

Por que não fornecer essas informações no próprio momento da criação?

Uma criação que declara os dados necessários

```
ItemPedido item =  
    new ItemPedido("Teclado", 150.0, 2);
```

Descrição, preço e quantidade aparecem na própria expressão de criação.

A classe precisa receber esses valores

```
public ItemPedido(String descricaoRecebida,  
                  double precoRecebido,  
                  int quantidadeRecebida) {  
    descricao = descricaoRecebida;  
    precoUnitario = precoRecebido;  
    quantidade = quantidadeRecebida;  
}
```

CONCEITO-CHAVE

CRIAR E INICIALIZAR

Construtor

Um construtor define como o estado inicial de um objeto é preparado no momento de sua criação.

A classe explicita aquilo de que o objeto precisa ao nascer.

Como reconhecer um construtor

```
public ItemPedido(String descricao,  
                  double precoUnitario,  
                  int quantidade) {  
    // preparação do estado inicial  
}
```

- possui o mesmo nome da classe;
- não declara tipo de retorno, nem `void` ;
- pode receber parâmetros;
- é executado durante a criação com `new` .

O que acontece com o código antigo?

```
ItemPedido item = new ItemPedido();
```

A classe agora declara apenas este construtor:

```
ItemPedido(String descricao,  
            double precoUnitario,  
            int quantidade)
```

Esse código ainda compila? Por quê?

ARMADILHA

IMPACTO DA MUDANÇA

“Mas `new ItemPedido()` funcionava antes”

Quando uma classe não declara construtor, Java fornece implicitamente um construtor sem argumentos.

Depois que declaramos o construtor com três parâmetros, essa forma implícita deixa de ser fornecida.

`new ItemPedido()` já não corresponde ao construtor disponível.

Criação e declaração precisam se encontrar

```
new ItemPedido("Teclado", 150.0, 2)
```

```
public ItemPedido(String descricaoRecebida,  
                  double precoRecebido,  
                  int quantidadeRecebida)
```

O CAMINHO DOS DADOS

Vamos acompanhar apenas um valor

150.0
argumento → **precoRecebido**
parâmetro → **precoUnitario**
campo

Três papéis diferentes

Argumento

valor ou expressão fornecida na chamada

Parâmetro

variável declarada para receber o valor

Campo

parte do estado em que o valor pode ser armazenado

argumento → parâmetro → campo

O construtor é o mesmo

```
ItemPedido teclado =  
    new ItemPedido("Teclado", 150.0, 2);  
  
ItemPedido mouse =  
    new ItemPedido("Mouse", 80.0, 3);
```

Cada execução recebe argumentos diferentes e prepara o estado de um novo objeto.

QUANDO OS NOMES COINCIDEM

Campo e parâmetro com o mesmo nome

```
public ItemPedido(String descricao,  
                  double precoUnitario,  
                  int quantidade) {  
    this.descricao = descricao;  
    this.precoUnitario = precoUnitario;  
    this.quantidade = quantidade;  
}
```

QUANDO OS NOMES COINCIDEM

Leia os dois lados

```
this.descricao = descricao;
```

THIS.DESCRICAO

campo do objeto atual

DESCRICAO

parâmetro recebido

DICA

QUANDO OS NOMES COINCIDEM

Siga o valor

150.0
argumento → **precoUnitario**
parâmetro → **this.precoUnitario**
campo

ATIVIDADE

ACOMPANHE A CRIAÇÃO

Antes de responder

```
new ItemPedido("Mouse", 80.0, 3)
```

1. Qual será o estado do objeto ao final do construtor?
2. O que aconteceria com `descricao = descricao; ?`
3. A qual `descricao` cada lado se refere?

ACOMPANHE A CRIAÇÃO

Estado ao final do construtor

Campo	Valor
descricao	"Mouse"
precoUnitario	80.0
quantidade	3

O caminho completo foi: argumento → parâmetro → campo.

ARMADILHA

ACOMPANHE A CRIAÇÃO

descricao = descricao;

```
public ItemPedido(String descricao) {  
    descricao = descricao;  
}
```

Os dois nomes se referem ao parâmetro.

O campo permanece sem receber o valor. `this.descricao` torna explícito o campo do objeto atual.

Agora exigimos todos os argumentos

```
ItemPedido item =  
    new ItemPedido("", -100.0, -4);
```

Receber todos os dados significa nascer em um estado aceitável?

ATIVIDADE

DADOS COMPLETOS?

Examine o estado proposto

```
new ItemPedido("", -100.0, -4)
```

- os três argumentos foram fornecidos;
- os três tipos estão corretos;
- o objeto deveria incorporar esses três valores?

DADOS COMPLETOS?

O tipo não conhece a regra do problema

Java

-100.0 é um double

-4 é um int

ItemPedido

preço e quantidade negativos não
representam estados adequados

Inicializar os campos não garante, por si só, que os valores façam sentido.

REGRAS DESTA ETAPA

Duas invariantes numéricas simples

- `precoUnitario >= 0`
- `quantidade >= 0`

O valor `0` permanece aceito.

A validação textual da descrição não será aprofundada agora.

O construtor avalia os valores

```
public ItemPedido(String descricao,  
                  double precoUnitario,  
                  int quantidade) {  
    this.descricao = descricao;  
  
    if (precoUnitario >= 0) {  
        this.precoUnitario = precoUnitario;  
    }  
  
    if (quantidade >= 0) {  
        this.quantidade = quantidade;  
    }  
}
```

Depois desta criação...

```
ItemPedido item =  
    new ItemPedido("Teste", -10.0, -2);
```

Quais valores ficam nos campos?

- descricao = ?
- precoUnitario = ?
- quantidade = ?

Os valores negativos não entram

Campo	Estado final
descricao	"Teste"
precoUnitario	0.0
quantidade	0

Quando a condição falha, o campo numérico permanece com seu valor padrão.

Preservar a regra não comunica a falha

```
new ItemPedido("Teste", -10.0, -2)
```

- o objeto não incorpora números negativos;
- preço e quantidade permanecem em zero;
- quem chamou não recebe uma informação sobre a rejeição.

Hoje o foco é preservar o estado. Outras políticas exigem repertório que ainda não introduzimos.

ATIVIDADE

ONDE A REGRA DEVE FICAR?

Proposta A — cada cliente verifica

Um programa recebe os dados de um item de pedido. Aqui, **cliente** é qualquer trecho do sistema que tenta criar esse objeto.

```
double precoInicial = 0.0;
if (preco >= 0) precoInicial = preco;

int quantidadeInicial = 0;
if (quantidade >= 0) quantidadeInicial = quantidade;

ItemPedido item =
    new ItemPedido(descricao, precoInicial, quantidadeInicial);
```

O que acontece quando outro cliente esquece essa verificação?

ATIVIDADE

ONDE A REGRA DEVE FICAR?

Proposta B — a classe preserva a regra

O mesmo trecho do sistema apenas repassa os dados recebidos:

```
ItemPedido item =  
    new ItemPedido(descricao, preco, quantidade);
```

Considere que o construtor valida preço e quantidade.

1. Qual proposta concentra a regra em todos os pontos de criação?
2. Quem conhece melhor os estados aceitáveis de `ItemPedido` ?

A regra acompanha o objeto

**qualquer cliente fornece valores → o construtor avalia →
o estado preserva as regras**

A criação e as mudanças posteriores pertencem a momentos diferentes, mas a responsabilidade continua com a classe.

Invariante

Uma invariante é uma regra que deve permanecer verdadeira para o estado do objeto.

- `precoUnitario >= 0`
- `quantidade >= 0`

A classe protege essas regras durante a criação e nas mudanças posteriores.

A ideia continua em outro domínio?

Em um sistema de hospedagem, cada `Reserva` registra a quantidade de pessoas e o valor de uma diária, nessa ordem:

```
Reserva reserva = new Reserva(4, 180.0);
```

Neste exemplo: `4` pessoas e diária de `R$ 180,00`.

1. Quais informações uma reserva precisa receber na criação?
2. `-2` pessoas deveria fazer parte do estado?
3. Uma diária negativa deveria fazer parte do estado?
4. Quem deveria proteger essas regras?
5. Que resultado rápido mostraria que a proteção funcionou?

O domínio mudou. A responsabilidade não.

ItemPedido

protege preço e quantidade

Reserva

deveria proteger pessoas e diária

Evidência rápida: tentar criar uma reserva com valor negativo e observar que esse valor não foi incorporado ao estado.

Os clientes fornecem valores; a classe conhece e preserva as regras de seu estado.

SÍNTESE
FECHAMENTO

Da criação vazia ao estado inicial coerente

o objeto podia nascer incompleto → o construtor exige os dados →
a classe protege invariantes

O que precisamos conseguir explicar

- por que configurar o objeto em etapas permite estados incompletos;
- o caminho argumento → parâmetro → campo;
- por que `this.campo` e o parâmetro têm papéis diferentes;
- por que receber todos os dados não garante coerência;
- quem deve preservar as regras do estado inicial.

PRÓXIMO PASSO

No Laboratório 05

Vamos evoluir a Versão 4 do Projeto 1:

- criar objetos com descrição, preço e quantidade;
- adaptar os pontos que ainda usam criação vazia;
- proteger todos os campos;
- preservar as invariantes também durante a criação.